SQL Data Types

SQL Data Types

Definition

SQL Data Types define the kind of values that can be stored in a column of a table. They ensure that data is stored in a consistent format and optimize database performance. Different database systems (MySQL, PostgreSQL, SQL Server, Oracle) may have variations in supported data types.

1. Numeric Data Types

Description

Numeric types store numbers. They can be **exact** (integers, fixed-point) or **approximate** (floating-point).

Common Numeric Types

- `INT`, `INTEGER` – Whole numbers (e.g., `INT` stores $-2,147,483,648$ to $2,147,483,647$).
- `SMALLINT` – Smaller range integer.
- `BIGINT` – Larger range integer.
- `DECIMAL(p,s)` / `NUMERIC(p,s)` – Exact fixed-point numbers with precision `p` and scale `s`.
- `FLOAT`, `REAL`, `DOUBLE PRECISION` – Approximate floating-point values.

Example

```
CREATE TABLE product (
    product_id INT PRIMARY KEY,
    price DECIMAL(8,2),  -- 8 digits total, 2 after decimal
    rating FLOAT
);
```

2. Character/String Data Types

Description

String types store text data. They can be fixed-length or variable-length.

Common String Types

- `CHAR(n)` – Fixed-length string of length `n`.
- `VARCHAR(n)` – Variable-length string with maximum length `n`.
- `TEXT` – Large variable-length text.

Example

```
CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    gender CHAR(1) CHECK (gender IN ('M', 'F'))
);
```

3. Date and Time Data Types

Description

Date/Time types store calendar dates and clock times.

Common Date/Time Types

- `DATE` – Stores year, month, and day.
- `TIME` – Stores hours, minutes, and seconds.
- `TIMESTAMP` – Stores both date and time.
- `INTERVAL` – Stores a span of time.

Example

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    order_date DATE,
    delivery_time TIME,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4. Boolean Data Type

Description

The `BOOLEAN` type stores logical values: `TRUE`, `FALSE`, or `NULL`.

Example

```
CREATE TABLE tasks (
    task_id INT PRIMARY KEY,
    task_name VARCHAR(100),
    is_completed BOOLEAN DEFAULT FALSE
);
```

5. Binary Data Types

Description

Binary types store raw byte data, such as images or files.

Common Binary Types

- `BINARY(n)` – Fixed-length binary data.
- `VARBINARY(n)` – Variable-length binary data.
- `BLOB` – Large binary data objects.

Example

```
CREATE TABLE files (
    file_id INT PRIMARY KEY,
    file_name VARCHAR(255),
    file_data BLOB
);
```

Key Notes

- Always choose the smallest data type that fits the data to optimize storage and performance.
- Different DBMSs may use different names or ranges for the same type.
- Use `CHECK` constraints to further restrict values beyond their data type.

2. Type Conversion and Formatting Functions

Definition

Type conversion is the process of changing a value from one data type to another. It can be **implicit** (automatic) or **explicit** (using functions like `CAST` or `CONVERT`).

Examples of Type Conversion

```
-- Explicit conversion using CAST
SELECT CAST('2025-08-12' AS DATE);

-- Explicit conversion using CONVERT (SQL Server)
SELECT CONVERT(INT, 12.34);

-- Formatting date output
SELECT TO_CHAR(SYSDATE, 'DD-Mon-YYYY') FROM dual; -- Oracle
```

3. Default Values

Definition

Default values in SQL specify an initial value for a column when no explicit value is provided during insertion.

Example: Default Values

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    join_date DATE DEFAULT CURRENT_DATE,
    status VARCHAR(10) DEFAULT 'Active'
);
```

4. Large-Object Types (LOBs)

Definition

Large Object (LOB) data types are used to store large amounts of data such as text, images, audio, and video. Common types include:

- `CLOB` – Character Large Object for large text data.
- `BLOB` – Binary Large Object for binary data.
- `NCLOB` – For large Unicode text data.

Example: LOB Usage

```
CREATE TABLE documents (
    doc_id INT PRIMARY KEY,
    doc_name VARCHAR(100),
    doc_content CLOB,
    doc_file BLOB
);
```

5. User-Defined Types (UDTs)

Definition

User-defined types allow database designers to create custom data types for domain-specific requirements. They can be based on existing SQL types with additional constraints.

Example: Creating a UDT

```
-- PostgreSQL Example
CREATE DOMAIN currency AS NUMERIC(12,2)
    CHECK (VALUE >= 0);

CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    price currency
);
```

6. Summary Table of Common SQL Data Types

Data Type Overview

Category	Data Types	Examples
Numeric	INT, BIGINT, DECIMAL, FLOAT	`123`, `45.67`
Character	CHAR(n), VARCHAR(n), TEXT	`'Hello'`
Date/Time	DATE, TIME, TIMESTAMP	`'2025-08-12'`
Binary	BLOB, BYTEA	Image data
Boolean	BOOLEAN	`TRUE`, `FALSE`
Large Objects	CLOB, NCLOB, BLOB	Documents, Media

Index Creation in SQL

Definition

An **index** is a database object that improves the speed of data retrieval operations on a table at the cost of additional storage and maintenance overhead. Indexes work like pointers to data in tables, allowing faster search and query performance.

Purpose of Indexes

- **Faster Retrieval:** Significantly improves the performance of `SELECT` queries.
- **Efficient Searching:** Optimized lookups using key values.
- **Sorting Assistance:** Speeds up queries with `ORDER BY` and `GROUP BY`.
- **Enforcing Uniqueness:** Unique indexes prevent duplicate values in columns.

Types of Indexes

Types

- **Single-Column Index:** Index on one column.
- **Composite Index:** Index on multiple columns.
- **Unique Index:** Ensures all indexed values are distinct.
- **Clustered Index:** Physically rearranges the table data to match index order.
- **Non-Clustered Index:** Maintains a separate structure from the table data.

Creating an Index

Basic Index Creation

Create an index on the `student` table for the `name` column:

```
CREATE INDEX idx_student_name
ON student(name);
```

Composite Index

Create an index on two columns `course_id` and `semester` in the `section` table:

```
CREATE INDEX idx_section_course_sem
ON section(course_id, semester);
```

Unique Index

Create a unique index to prevent duplicate email addresses:

```
CREATE UNIQUE INDEX idx_unique_email
ON student(email);
```

Dropping an Index

Drop Index

Drop an index when it is no longer needed:

```
DROP INDEX idx_student_name;
```

Best Practices

Best Practices

- Index columns used frequently in `WHERE`, `JOIN`, `ORDER BY`.
- Avoid excessive indexing to reduce storage and maintenance costs.
- Monitor query performance using `EXPLAIN` plans.
- Use **composite indexes** wisely — order matters.

Domains in SQL

Definition

A **domain** in SQL defines the permissible set of values for a column. It is essentially a named data type with optional constraints that can be reused across multiple tables. Domains help enforce data consistency and integrity.

Purpose of Domains

- **Consistency:** Ensures the same rules are applied wherever the domain is used.
- **Reusability:** A domain can be used in multiple tables without redefining constraints.
- **Maintainability:** Updating the domain updates all dependent columns.
- **Validation:** Prevents invalid data from being inserted.

Creating a Domain

Basic Domain Creation

Create a domain for storing positive integer IDs:

```
CREATE DOMAIN positive_id AS INTEGER
CHECK (VALUE > 0);
```

Domain with String Constraint

Create a domain for storing country codes:

```
CREATE DOMAIN country_code AS CHAR(2)
CHECK (VALUE ~ '^[A-Z]{2}$');
```

Using a Domain in a Table

Applying a Domain

Use the `positive_id` domain in a table definition:

```
CREATE TABLE student (
    student_id positive_id PRIMARY KEY,
    name VARCHAR(50)
);
```

Altering and Dropping Domains

Alter Domain

Add a default value to the `country_code` domain:

```
ALTER DOMAIN country_code
SET DEFAULT 'US';
```

Drop Domain

Remove a domain (only if no column is using it):

```
DROP DOMAIN country_code;
```

Best Practices

Best Practices

- Use domains for commonly reused data types with rules.
- Name domains descriptively (`email_type`, `price_currency`).
- Keep domain constraints clear and maintainable.
- Avoid overly complex constraints that reduce flexibility.